Data Structures

Practical 3

Aim: To implement a stack and operations on stack

Theory

Stack is a linear data structure that follows a particular order in which the operations are performed. The order may be **LIFO(Last In First Out)**. Data element inserted last is removed first and operations used are called push and pop. Data is added and removed from one end of stack and it is called stack top. One pointer points to top element of the stack. Whenever element is added, it is added on the top of the stack. And element is removed from the same side, top of the stack.

It behaves like a real world stack. We can imagine example of stack as pile of plates, where new plate placed on top of the pile and plate is removed one which is on top. Similarly, stack data structure adds new element on top of the stack and deletes an element from top.

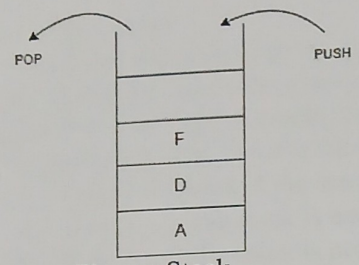


Figure: Stack

This strategy states that the element that is inserted last will come out first. You can take a pile of plates kept on top of each other as a real-life example. The plate which we put last is on the top and since we remove the plate that is at the top, we can say that the plate that was put last comes out first.

Basic operations used in stack are:

- push() – Pushing (inserting) an element in the stack.
- pop() – Pop out (removing) an element from the stack.

If stack contains no items in it, that is called empty stack. Pop operation cannot be applied to emty stack as there is no data item to pop. If stack is full, there is no space to insert into it, push operation is not applicable. When data is added into stack with PUSH operation, we need to check for empty space to add an element. And to remove data from the stack, check whether data is present or stack is empty. To use a stack efficiently, we need to check the status of stack as well. For the same purpose, the following functionality is added to stacks –

- peek() – get the top data element of the stack, without removing it.
- isFull() – check if stack is full.
- isEmpty() – check if stack is empty.

At all times, we maintain a pointer to the last PUSHed data on the stack. As this pointer always represents the top of the stack hence named top. The top pointer provides top value of the stack without actually removing it.

Push: Adds an item to the stack. If the stack is full, then it is said to be an Overflow condition.

Algorithm for push:

Steps for PUSH() operation:

Step 1 – Check if the stack is full.
Step 2 – If the stack is full, produces an error and exit.
Step 3 – If the stack is not full, increments top to point next empty space.
Step 4 – Adds data element to the stack location, where top is pointing.
Step 5 – Returns success.

If the linked list is used to implement the stack, then in step 3, we need to allocate space dynamically.

Algorithm for PUSH Operation

A simple algorithm for Push operation can be derived as follows –

begin procedure push: stack, data
if stack is full

return null
endif
top ← top + 1
stack[top] ← data
end procedure

Implementation of this algorithm in C,

```
void push(int data) {
if(!isFull()) {
top = top + 1;
stack[top] = data;
} else {
printf("Could not insert data, Stack is full.\n");
}
}
```

Pop() Operation

Accessing the content while removing it from the stack, is known as a pop Operation. In an array implementation of pop( ) operation, top is decremented to a lower position in the stack to point to the next value. But in linked-list implementation, pop() actually removes data element and deallocates memory space.

A Pop operation may involve the following steps −

- Step 1 − Check if the stack is empty.
- Step 2 − If the stack is empty, produces an error and exit.
- Step 3 − If the stack is not empty, accesses the data element at which top is pointing.
- Step 4 − Decrease the value of top by 1.
- Step 5 − Return success.

Algorithm for Pop Operation

begin procedure pop: stack
if stack is empty
return null
endif
data ← stack[top]
top ← top - 1
return data
end procedure

Implementation of this algorithm in C, is as follows −

```
int pop(int data) {
if(!isempty()) {
data = stack[top];
top = top - 1;
return data;
} else {
printf("Could not retrieve data, Stack is empty.\n");
}
}
```

peek() operation:

Algorithm of peek() function −
begin procedure peek
return stack[top]
end procedure

Implementation of peek() function in C programming language –

```
int peek()
{
```

```
return stack[top];
}
```

isfull() operation :

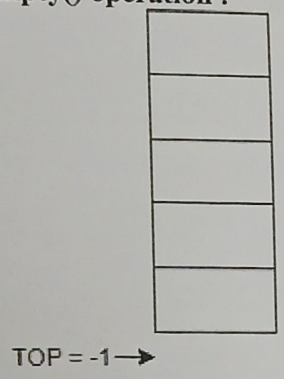
Algorithm –

```
begin procedure isfull
if top = MAXSIZE
return true
else
return false
endif
end procedure
```

Implementation of isfull() function in C programming language –

```
int isfull()
{
  if (top == MAXSIZE)
  return true;
  else
  return false;
}
```

isempty() operation :



stack: st

Algorithm –

```
begin procedure isempty
if top < 1
return true
else
return false
endif
end procedure
```

Implementation of isempty() function in C programming language is slightly different. We initialize top at -1, as the index in array starts from 0. So we check if the top is below zero or -1 to determine if the stack is empty.

```
int isempty()
{
  if(top == -1)
    return true;
  else
    return false;
}
```

```
/*PracticalNo.03
    Title: Write a program to implement stack and operations on stack.
*/

#include<stdio.h>
#include<conio.h>
#define maxsize 10
void push(struct stack *p, int x);
int pop(struct stack *ps);
struct stack
{
  int top;
  int item[maxsize];
}s;
void main()
{
      int choice, ele, *i;
      struct stack *ps;
      s.top = -1;
      printf("Select one option: \n Choice1: PUSH \n Choice2:POP \nChoice3: Display stack \n Choice 4:
      Exit \n");
      while(choice!=4)
      {
          printf("\nEnter correct option:");
          scanf("%d", &choice);
          if (choice==1)
          {
              printf("\nEnter the element to be inserted:");
              scanf("%d", &ele);
              push(&s, ele);
          }
          if(choice==2)
          {
              *i=pop(&s);
              printf("\nThe deleted item is:%d\n", *i);
          }
          if(choice==3)
          {
              showstack(&s);
          }
          if(choice==4)
          {
              exit(0);
          }
      }
      getch();
}
void push(struct stack *ps, int x)
{
      Ps->top++;
      Ps->item[ps->top]=x;
}

int pop(struct stack *ps)
{
      int y;
      y=ps->item[ps->top];
      ps->top--;
```

```
        return y;
}

void showstack(struct stack *ps)
{
        int i;
        if(ps->top==-1)
        {
            printf("Stack is empty…");
        }
        else
        {
            printf("Elements of stack:-\n");
            for(i=ps->top; i>=0; i--)
            {
                printf("%d\n",ps->item[i]);
            }
        }
}
```

OUTPUT:

Choice 1:PUSH
Choice2:POP
Choice 3:Display stackChoice4:Exit
Enter your choice:1
Element to be inserted:4
Enter your choice:1
Element to be inserted:7
Enter your choice:1
Element to be inserted:9
Enter your choice:3
Elements of stack:-
9
7
4
Enteryourchoice:4